

Week 15

Distributed Training with PyTorch; torchrun and accelerate

- Welcome to the Advanced Phase
- This week, we unlock scalable, high-value AI workflows
- Hands-on with real industry tools for multi-GPU training

Learning Outcomes

- Understand distributed training in machine learning
- Explain data parallelism and its benefits
- Use torchrun and accelerate for multi-GPU workflows
- Monitor and optimise distributed ML jobs
- Apply skills in real cloud and HPC environments

Why Distributed Training Matters

- Speeds up AI model training for large datasets and complex models
- Enables solving industry-scale ML problems
- Prepares you for real-world roles in AI development and cloud computing

Distributed Training. Fundamentals

- Splits model training work across multiple devices or GPUs
- Makes use of parallel processing for faster, more scalable AI results
- Standard in enterprise AI and ML engineering

Data Parallelism. Key Concept

- Dataset is divided across workers; each processes a subset
- Workers train the same model instance on different data slices
- Results are averaged or combined for overall learning
- Example; Training a vision model across four GPUs in parallel

Single-GPU vs Multi-GPU Workflows

Feature	Single-GPU	Multi-GPU (Distributed)
Performance	Slower; sequential	Faster; parallelised
Model Size Limit	Limited to GPU memory	Much larger models possible
Common Use	Prototypes and small models	Industry workflows; production
Complexity	Simpler	Needs extra configuration

Meet torchrun

- Built-in PyTorch utility for launching distributed jobs
- Simplifies multi-GPU setup and execution
- Used in cloud VMs and HPC environments

Basic torchrun Example

```
torchrun --nproc_per_node=2 train.py
```

- Launches train.py script using two GPUs
- Handles communication and process management
- Common issues; incorrect device count, environment misconfiguration

Accelerating with accelerate

- High-level library by Hugging Face for distributed ML
- Automates device placement, synchronisation, and scaling
- Widely used in industry for research, production, and open-source projects

accelerate Workflow Example

- Init your script with accelerate CLI
- Handles hardware detection and resource sync
- Example;

```
accelerate config  
accelerate launch train.py
```

- Reduces boilerplate; makes distributed training accessible

Synchronisation and Performance Tips

- Always check GPU resources before launch
- Avoid data bottlenecks by using efficient dataloaders
- Monitor synchronisation between devices; stragglers slow everyone
- Use experiment tracking to compare runs

Industry Example. AI Model Training at Scale

- Local AI company trains language models on Azure cloud with eight GPUs
- Uses torchrun for distributed scripts and experiment tracking tools
- Achieves week-long training jobs in a few hours; aligns with industry standards

Practice; Distributed PyTorch with torchrun

- Clone or use your notebook with torchrun compatibility
- Run a simple training script across two or more GPUs
- Compare speed and resource use to single-GPU runs
- Document your steps and findings for portfolio

Practice; accelerate for Distributed ML

- Set up accelerate in your cloud or VM workspace
- Launch a distributed training script using accelerate CLI
- Monitor job logs and system performance
- Reflect on ease of use compared to torchrun

Assessment Preparation

- Week 17 project; Run and monitor a multi-GPU PyTorch script
- Use torchrun or accelerate, keep experiment records
- Track performance, synchronisation, and resource efficiency
- Start documenting setup steps for your assessment portfolio

Summary and Next Steps

- Distributed training is essential for AI at scale
- torchrun and accelerate support efficient, high-value ML workflows
- Apply today's skills in next week's hands-on project
- Bring questions and documentation for assessment readiness

